



Multi-objective Optimization of Ticket Assignment Problem in Large Data Centers

Tariq Ali Arain, Xiangjie Huang, Zhicheng Cai^(✉), and Jian Xu

School of Computer Science and Engineering,
Nanjing University of Science and Technology, Nanjing, China
caizhicheng@njjust.edu.cn

Abstract. Software or hardware problems in large data centers are usually packaged to be tickets which are assigned to different experts to solve. It is very crucial to design multi-objective ticket scheduling algorithms to maximize the total matching degree and minimize the total flowtime. However, most of existing methods for assignment problems only consider single objective, while some methods optimizing multi-objectives are not for the same objectives of this paper. Meanwhile, exploring effectiveness of existing meta-heuristics for multi-objective optimization could be improved further. In this paper, a multi-objective heuristic algorithm called (GAMOA*) is proposed for ticket scheduling which is the combination of a genetic algorithm (GA) and a multi-objective A* (MOA*). In GAMOA*, ticket scheduling orders are evaluated and improved by GA, while MOA* is applied to find a Pareto set of solutions given an order of tickets effectively and efficiently. Experimental results illustrate that our approach obtains better results than state-of-art algorithms.

Keywords: Genetic algorithm · Ticket scheduling · Multi-objective A star · Cloud Computing · Routing problem

1 Introduction

End-users and application developers using Cloud Computing facilities may meet software or hardware problems inevitably. User problems are usually recorded to be tickets (contain problem descriptions) and sent to back-end experts to solve. From the perspective of Cloud Computing providers, solving these problems efficiently and effectively is beneficial to improving user experiences. Experts have diverse technique background and problem solving experience, then have different matching degrees for a specific ticket which determine the quality of solving the tickets. Meanwhile, experts also have different estimated processing times for the same ticket which have great impact on the response time to users. Therefore, it is crucial to develop ticket scheduling algorithms in terms of matching degrees and processing times.

The multi-objective scheduling of tickets to experts is an NP-hard problem. In this paper, the total matching degree and total flowtime of overall tickets

are optimized simultaneously where the flowtime of a ticket is the finish time minus the arrival time of it. This ticket scheduling problem can be modeled to be a multi-objective assignment problem which is a well known NP-hard problem [1]. Most of existing works for assignment problems address the optimization of a single objective function [2,3]. The multi-objective optimization can not be solved by these methods directly [4]. Meanwhile, there are only a few existing algorithms for solving multi-objective assignment problems using genetic algorithms, simulated annealing and particle swarm optimization algorithms [1,5,6]. However, the exploring ability of these multi-objective meta-heuristic algorithms can be enhanced by some exact algorithms like A star algorithm.

In this article, a hybrid of genetic algorithm and multi-objective A* (called GAMOA*) is proposed for the ticket scheduling problem by taking advantages of genetic algorithms and A* simultaneously. The genetic algorithm first sorts out all tickets randomly to produce different ticket scheduling orders called genetic chromosomes. Afterward, scheduling ordered tickets to experts is transformed into a routing problem which can be solved by multi-objective A* effectively. Next, solutions found by the multi-objective A* (MOA*) is used to calculate the fitness of each chromosome, based on which better chromosomes are generated by crossover and mutations. Finally, new chromosomes are evaluated by MOA* again and the above process iterates until the stopping criteria are met. The main contributions include (1) a novel multi-objective heuristic algorithm called (GAMOA*) is proposed which is the hybrid of genetic algorithm and multi-objective A*. (2) a Pareto-set based fitness calculation method is developed for the individual with multiple solutions and a transforming method is proposed to transform the ticket scheduling problem into a routing problem which is the basis of using multi-objective A* as a local search method.

The rest of the article is organized as follows: Sect. 2 is related work. Section 3 demonstrates the problem formulation. The proposed heuristic is described in Sect. 4. Experimental setup and results are presented in Sect. 5. Finally, Sect. 6 concludes the paper.

2 Related Work

Ticket routing and ticket scheduling are two important problems about ticket solving. Ticket routing focus on finding the right order of candidate experts to assign each ticket. Tickets are transferred to next expert on the path when they can not be solved by the current experts. On the contrary, ticket scheduling is in charge of finding an appropriate assignment considering both the total matching degree (between tickets and experts) and the total flowtime. Ticket routing techniques could be used to enhance ticket scheduling.

2.1 Ticket Routing

When a client's request reaches the help desk, the operator has to view it, categorize it, and forward it to the responsible expert to investigate and fix the requested problem. The procedure is termed ticket routing. The ticket scheduling

to the experts has a resemblance with the ticket routing. Ticket content-mining [7] and historical sequence [8, 9] based methods have been developed for the ticket routing. Shao et al. [8] developed a route recommender system based on different orders of Markov models established from historical processing sequence data. In content-based methods [7], tickets are usually classified to be different categories by analysing text descriptions based on diverse machine learning techniques. Content-based methods are usually only better on the tickets with rich text descriptions. However, for general tickets with not too much descriptions, it has been proved that content-based methods are usually outperformed by sequence based methods [9]. There are also some hybrid methods based on Markov model and content-mining [9]. However, most of existing methods about ticket solving primarily focus on ticket routing ignoring the optimization of workload distribution among experts.

2.2 Ticket Scheduling

Many methods have been developed for different kinds of assignment problems and most of them are designed for single objective. Discrete particle swarm optimization (DPSO) based methods are extensively used in multi-objective optimization of assignment problems such as task scheduling in Grid [10], load balancing in heterogeneous computing systems [11, 12] and task scheduling in a distributed computing system [13, 14] in terms of makespan, flowtime, reliability and cost objectives, etc. Genetic algorithms are also widely used to solve multi-objective optimization of assignment problems. Subhashini et al. [15, 16] developed a hybrid method of Elitist Non-dominated Sorting Genetic Algorithm (NSGA-II) and the Non-dominated Sorting Particle Swarm Optimization Algorithm (NSPSO) to schedule tasks in a heterogeneous environment for minimizing the makespan and flowtime. Khodadadi et al. [17] proposed an Genetic Algorithm with Variable Neighborhood Search (GA-VNS) for independent task scheduling in a grid environment to reduce the total cost and makespan. Many other nature-inspired meta-heuristics are also applied for assignment problems such as Simulated Annealing (SA) [18], Ant Colony Optimization (ACO) [18], and Cuckoo and Gravitational Search (CGSA) [19]. However, there is no algorithm designed for the ticket scheduling of this paper.

In a word, existing algorithms are usually designed for single objectives or not for the multi-objective ticket scheduling. Meanwhile, existing meta-heuristics could be improved by more efficient and effective local search methods because the convergency speed of traditional meta-heuristic algorithms are very slow for large scale ticket scheduling problems.

3 Problem Description

$E = \{e_i, i = 1, \dots, N\}$ is the set of back-end experts in the Cloud Computing Data Centers. It is assumed that tickets are scheduled to experts periodically. For example, tickets are collected to be a batch and scheduled to experts every six hours. It is assumed that all experts have finished the processing of the

previous batch and are idle currently. $T = \{t_j, j = 1, \dots, K\}$ is the set tickets in the current batch. p_{ij} and m_{ij} denotes the processing time and matching degree of ticket t_i on expert e_j respectively. Multiple tickets can be assigned to the same expert. However, an expert cannot start a new ticket before finishing previous tickets. Let $d_{i,j}$ be the index of ticket i on expert j . If ticket i is not assigned on expert j , $d_{i,j} = 0$. For example, when tickets t_1 , t_2 and t_3 are assigned on expert e_1 one by one, $d_{1,1} = 1$, $d_{2,1} = 2$ and $d_{3,1} = 3$. The flowtime $f_{i,j}$ of a ticket t_i on expert e_j is its own processing time plus the waiting time (used to process previous tickets). The objective of this article is to minimize the total flowtime and maximize the total matching degree. The mathematical model of this multi-objective tickets schedule is as follow.

$$\max \sum_{i=1}^K \sum_{j=1}^N m_{i,j} \cdot (d_{i,j} > 0) \quad (1)$$

$$\min \sum_{i=1}^K \sum_{j=1}^N f_{i,j} * (d_{i,j} > 0) \quad (2)$$

$$\sum_{j=1}^N (d_{i,j} > 0) = 1, i = 1, \dots, K \quad (3)$$

$$f_k = \sum_{i=1}^K \sum_{j=1}^N p_{i,j} (d_{i,j} > 0) * (d_{i,j} \leq d_{k,j}), k = 1, \dots, K \quad (4)$$

Equation (1) and (2) maximize the total matching degree and minimize the total flowtime. Equation (3) represents that one ticket is assigned to only one expert. Equation (4) illustrates that the flowtime of ticket t_k is the processing time of t_k plus processing times of tickets before t_k on the same expert.

4 Proposed GAMOA*

In this paper, a genetic algorithm-based multiple-objective A star algorithm (GAMOA*) is presented for the multi-objective ticket scheduling problem. Genetic algorithms (GA) have been widely used to solve multiple-objective scheduling problems and obtain good performance. The main reason is that GA has the ability to exploring the search space efficiently. However, the traditional GA lacks the ability to explore local optimal areas deeply. Therefore, some local search methods have been reported to aid GA. Nonetheless, an exhausted local search is very time-consuming. A star is an effective and efficient method for route searching. In this article, the ticket scheduling is transformed into a routing problem and solved by a multi-objective A star (MOA*) as an effective local search method of GA. Figure 1a explains the flow chart of the proposed GAMOA*. First, ticket orders are generated via a genetic algorithm. Second, the fittest orders are selected via the fitness function. Then ticket assignment on experts with the given ticket orders is transformed into a routing problem.

Next, the routing problem is solved by the MOA* to find assignments of tickets on experts in detail. Finally, a set of Pareto solutions is obtained for each order of tickets. The fitness of this ticket order is evaluated based on these solutions in terms of the total flowtime and matching degree. The above process iterates until the stopping criteria are met.

4.1 Transforming Ticket Scheduling to Routing

Traditional routing problems are usually modeled via a directed graph with weight on edges. The goal of routing algorithms is to find the route from source to sink nodes with the optimal objectives. However, the goal of ticket scheduling is to find an assignment of tickets to experts. Tickets can be assigned to experts in any order. The A-star algorithm cannot be used to solve the original ticket scheduling, which is very different from the routing problem. However, when the ticket scheduling order is given, the ticket scheduling can be transformed into a routing problem as follows: For example, in the left part of Fig. 1b, tickets are ordered to be $\{T_3, T_1, T_2, T_4\}$ which means that they are scheduled from the top to bottom. On each row, nodes on the right of the ticket are candidate experts for that ticket. The same expert might appear on multiple rows, which means that multiple tickets can be assigned to the same expert. The calculating of a ticket's flowtime should consider already assigned tickets before this ticket. Experts of different levels are connected by up-down links, and a source and sink nodes are added to construct a directed graph. A route from the source node to the sink node on the graph means a schedule of tickets to experts. For each level, different selected nodes on the route mean assigning the corresponding tickets to different experts. For this routing problem, the A* algorithm can be used to solve it effectively. However, different orders of tickets have a significant impact on the A* for the above routing problem.

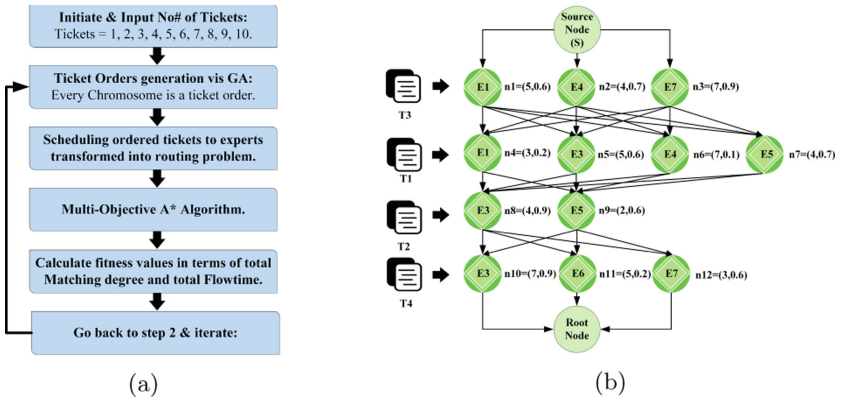


Fig. 1. An illustration of architecture of the proposed GAMOA* and transforming of assignment problems into routing problem

4.2 Multi-objective A* for Scheduling Ordered Tickets

Based on the proposed estimation function, the existing multi-objective A* (MOA*) [20] is applied to solve the above-mentioned routing problem. MOA* applies a best-first search technique which uses $\vec{F}_n = \vec{g}_n + \vec{h}_n$ as an evaluation method to select the next open node for expansion where $g(n)$ keeps the values (in tuples of multiple objectives) of the routes from the source node to the current node in the search graph and $h(n)$ estimates the path values (in tuples too) from the current node to the final destination node. The calculating methods of $h(n)$ determine the computation time of MOA* because $f(n)$ is used to cut searching branches. In this paper, an appropriate $g(n)$ function is found to reduce the time while maintaining search effectiveness.

Let $(t_{w_1}, t_{w_2}, t_{w_3}, \dots, t_{w_k}, \dots, t_{w_K})$ be the ordered tickets and n is the node at the level of ticket t_{w_k} in the routing graph. $S_p = (n, \vec{g}_n, F(n, \vec{g}_n))$ is a partial solution from s to node n in the graph of the routing problem, where $F(n, \vec{g}_n)$ is the function to calculate $\vec{F}_n$ based on S_p . For each S_p , $\vec{g}_n$ and the path to obtain S_p is stored in $G_{op}(n)$ in the form of $\{(\vec{g}_n, P_{\vec{g}_n}^s)\}$ where $P_{\vec{g}_n}^s$ is the set of paths obtaining $\vec{g}_n$, like $\{(n1, n4, \dots, ni), (n2, n5, \dots, ni)\}$. $P_{\vec{g}_n}^s$ will be updated when a new path with the same $\vec{g}_n$ was extended. For one successor node m , $P_{\vec{g}_m}^s$ will be updated by adding paths $(P_{\vec{g}_n}^s[w], n), w = 1, \dots, |P_{\vec{g}_n}^s|$. In the partial solution S_p , the flowtime of ticket t_i on expert e_j is

$$f_{i,j} = \sum_{l=1}^K p_{l,j}(d_{l,j} > 0) * (d_{l,j} \leq d_{i,j}) \quad (5)$$

where $d_{l,j}$ is determined by $P_{\vec{g}_n}^s$. For each node n , there might be multiple partial solutions, i.e., $G_{op}(n)$ may have more than one elements with different $\vec{g}_n$. Meanwhile, multiple paths might have the same $\vec{g}_n$ making $P_{\vec{g}_n}^s$ have multiple paths. The calculation of $g(n)$ is the sum up of total flowtime and matching degree from source node to current node along the path of the current node as follows.

$$\vec{g}_n = \left(\sum_{i=t_{w_1}}^{t_{w_k}} f_{i,P_{\vec{g}_n}^s}[0][i], \sum_{i=t_{w_1}}^{t_{w_k}} m_{i,P_{\vec{g}_n}^s}[0][i] \right) \quad (6)$$

where $P_{\vec{g}_n}^s[0]$ means get the first path from $P_{\vec{g}_n}^s$ and $P_{\vec{g}_n}^s[0][i]$ returns the index of selected expert for ticket t_i . Let CE_i be the set of candidate experts' index of t_i . $h(\vec{n})$ is the sum-up of minimum flowtime and maximum matching degree of remaining unassigned tickets based on the partial solution. Flow times and matching degrees are calculated separately, which means that unassigned tickets are not necessarily scheduled to the same expert when calculating the minimum flowtime and matching degree. During the calculating of flowtime, execution times of tickets scheduled in the partial solution and tickets before the current ticket should be considered. Let $P_{\vec{g}_n}^{sf}$ and $P_{\vec{g}_n}^{sm}$ record partial paths to estimate

flowtime and matching degree based on $\vec{g}_n$, respectively.

$$\vec{h}_n = (\sum_{i=t_{w_{k+1}}}^{t_{w_K}} \min_{j \in CE_i} \{f_{i,j}\}, \sum_{i=t_{w_{k+1}}}^{t_{w_K}} \max_{j \in CE_i} \{m_{i,j}\}) \quad (7)$$

where $f_{i,j}$ is determined by $P_{\vec{g}_n}^{sf}$.

Algorithm 1. Multi-objective A* (MOA).

Input: Ticket order T_o ;

```

1: Initialize the source node ( $s$ ), destination node ( $e$ ), reached goal node set  $GOALN$ ,
   non-dominated cost set  $COSTS$ , the sets of non-dominated cost vectors of paths
   reaching  $n$  that have or have not been explored  $G_{cl}(n)$  and  $G_{op}(n)$ , further
   expanded path set  $OPEN$  and add  $S$  to set  $OPEN = (s, \vec{g}_s, F(s, \vec{g}_s))$ ;
2: while  $OPEN$  is not empty do
3:   Select non-dominated ( $n, \vec{g}_n, F(s, \vec{g}_n)$ ) with smallest flowtime in  $OPEN$ 
4:   Move  $\vec{g}_n$  from  $G_{op}(n)$  and add  $\vec{g}_n$  to  $G_{cl}(n)$  if  $\vec{g}_n$  not dominated by  $G_{cl}(n)$ ;
5:   if  $n$  is final node then
6:     Include  $n$  in  $GOALN$  and  $\vec{g}_n$  in  $COSTS$ ;
7:     Filtering  $F(x)$  in  $OPEN$ ;
8:     continue;
9:   else
10:    Get all successors nodes  $m$  and calculate  $\vec{g}_m = \vec{g}_n + \vec{c}(n, m)$ ;
11:    if  $m$  is new then
12:      Calculate  $F_m$  filtering estimates dominated by  $COSTS$ ;
13:      if  $F_m$  is not empty then
14:        Put  $(m, \vec{g}_m, F(m, \vec{g}_m))$  in  $OPEN$ 
15:         $P_{\vec{g}_m}^s \leftarrow (P_{\vec{g}_n}^s[w], n), w = 1, \dots, |P_{\vec{g}_n}^s|$ . And add  $(\vec{g}_m, P_{\vec{g}_m}^s)$  to  $G_{op}(m)$ 
16:        continue;
17:      end if
18:    else
19:      if  $\vec{g}_m \in G_{op}(m) \cup G_{cl}(m)$  then
20:         $P_{\vec{g}_m}^s \leftarrow (P_{\vec{g}_n}^s[w], n), w = 1, \dots, |P_{\vec{g}_n}^s|$ . And add  $(\vec{g}_m, P_{\vec{g}_m}^s)$  to  $G_{op}(m)$ 
21:        continue;
22:      else
23:        Calculate  $F_m$  filtering estimates dominated by  $COSTS$ ;
24:        if  $F_m$  is not empty then
25:          Put  $(m, \vec{g}_m, F(m, \vec{g}_m))$  in  $OPEN$ 
26:           $P_{\vec{g}_m}^s \leftarrow (P_{\vec{g}_n}^s[w], n), w = 1, \dots, |P_{\vec{g}_n}^s|$ . And add  $(\vec{g}_m, P_{\vec{g}_m}^s)$  to  $G_{op}(m)$ 
27:          continue;
28:        end if
29:      end if
30:    end if
31:  end if
32: end while
33: return Non-dominated results  $R$ ;

```

In the multi-objective A* algorithm, the *OPEN* set record the reachable path. If the *OPEN* set is not empty, the partial solution $S_p = (n, \vec{g}_n, F(n, \vec{g}_n))$ with the minimum flowtime is taken from *OPEN*, and corresponding $(\vec{g}_n, P_{g_n}^s)$ is removed from the unexplored set G_{op} and added to the explored set G_{cl} in the form of $\{(\vec{g}_n, P_{g_n}^s)\}$. If the fetched node n is the destination node, it is added to the goal node-set *GOALN* and the $\vec{g}_n$ value is added to the *COSTS* set. If it is not the destination node, then the reachable node is obtained. The extension node will be filtered if it is dominated by G_{op} and G_{cl} , and finally add the remaining extension nodes to *OPEN*. The formal description of multiple-objective A* (MOA*) algorithm is shown in Algorithm 1.

An Example of MOA*: The process of MOA* for scheduling ordered tickets in Fig. 2a is as follows. Triples of partial solutions and values of *GOALN* and *COSTS* are shown in Table 1.

- **Iteration-1:** $G_{op}(S) = (0, 0)$, and $G_{cl}(S) = \emptyset$. From the S node to the destination node, the flowtime and matching degree of T1, and T2 need to be calculated for calculating $h(S)$. For ticket T1, the earliest finish time is 4 on E2, and the maximum matching degree is 0.7 on E2. With T1 executed on E2, the earliest finish time of T2 is 3 on E1, and the maximum matching degree of T2 is 0.6 on E3. Therefore $h(S) = (7, 1.3)$. The only path in *OPEN* is selected, and its two extensions to nodes $n1$ and $n2$ are added to *OPEN*, as shown in Fig. 2a.
- **Iteration-2:** When calculating $h(n1)$, T1 is executed on E1. If T2 is executed on E1, the finish time of T2 is 8 including 5 min of waiting and 3 min of execution. The finish time of T2 is executed on E3 is 5. Therefore, the earliest finish time is 5 on E3. The maximum matching degree is 0.6 on E3. With T1 executed on E2, the earliest finish time of T2 is 3 on E1, and the maximum matching degree of T2 is 0.6 on E3. The value of $h(n1)$ and $h(n2)$ are (5, 0.6) and (3, 0.6) respectively. Update $OPEN = \{(n1, (5, 0.6), (10, 1.2)), (n2, (4, 0.7), (7, 1.3))\}$ and insert $((5, 0.6), (S))$ into $G_{op}(n1)$ and $((4, 0.7), (S))$ into $G_{op}(n2)$. The non-dominated $n2$ with lowest flowtime is selected.
- **Iteration-3,** update $OPEN = \{(n1, (5, 0.6), (10, 1.2)), (n3, (7, 0.9), (7, 0.9)), (n4, (9, 1.3), (9, 1.3))\}$. The reason why $h(n3) = h(n4) = (0, 0)$ is that the flowtime and matching degree from $n3$ and $n4$ to the destination node are both zero. Insert $((7, 0.9), (S, n2))$ into $G_{op}(n3)$ and $((9, 1.3), (S, n2))$ into $G_{op}(n4)$. And $((4, 0.7), (S))$ will be moved to $G_{cl}(n2)$ from $G_{op}(n2)$. $n3$ is non-dominated with smallest flowtime in *OPEN* and selected for the extension towards node “e”. The node “e” represents a dummy node, and $n3$ is added to the *GOALN* array. The cost vector of $n3$ is included in the *COSTS* array. The $n3$ is closed and $((7, 0.9), (S, n2))$ will be moved to $G_{cl}(n3)$ from $G_{op}(n3)$.
- **Iteration-4,** update $OPEN = \{(n1, (5, 0.6), (10, 1.2)), (n4, (9, 1.3), (9, 1.3))\}$. $n4$ is non-dominated with smallest flowtime in *OPEN* and selected for the extension towards node “e”. Update $GOALN = \{n3, n4\}$ and $COSTS = \{(7, 0.9), (9, 1.3)\}$.

- **Iteration-5**, update $OPEN = \{(n1, (5, 0.6), (10, 1.2))\}$. The node is selected for an extension to proceeds towards the goal nodes E1 and E3. The node E1 is closed and inserted into the $G_{cl}(n1)$.
- **Iteration-6**, the estimate $(n3, (13, 0.8), (13, 0.8)), (n4, (10, 1.2), (10, 1.2))$ will be filtered by $COSTS$ causing neither $n3$ nor $n4$ is added to $OPEN$. Update $OPEN = \emptyset$. All the alternate solutions are filtered. The $GOALN$ and $COSTS$ arrays are solution node $n3(7, 0.9), n4(9, 1.3)$.

Table 1. An example of MOA* OPEN ($<<< =$ selected node).

Iteration	OPEN	GOALN	COSTS
1	(s, (0, 0), (7, 1.3))	$\emptyset$	$\emptyset$
2	(n1, (5, 0.6), (10, 1.2)), (n2, (4, 0.7), (7, 1.3)) $<<< S$	$\emptyset$	$\emptyset$
3	(n1, (5, 0.6), (10, 1.2)), (n3, (7, 0.9), (7, 0.9)), $<<< n2$ (n4, (9, 1.3), (9, 1.3))	(n3)	(7, 0.9)
4	(n1, (5, 0.6), (10, 1.2)), (n4, (9, 1.3), (9, 1.3)) $<<< n2$	(n3, n4)	(7, 0.9), (9, 1.3)
5	(n1, (5, 0.6), (10, 1.2)) $<<< S$	(n3, n4)	(7, 0.9), (9, 1.3)
6	$\emptyset$	(n3, n4)	(7, 0.9), (9, 1.3)

4.3 Genetic Algorithm for Ticket Ordering

The number of ticket orders is exponential to the number of tickets. It is very time-consuming to solve every ticket orders using A*. In this article, a multiple-objective GA (MGA) is employed to find appropriate ticket orders. MOA* is used to collect the set of Pareto solutions for each individual.

Chromosome Encoding: Each chromosome is expressed as an array with a length of K which represents a ticket order. Each chromosomes gene represents the index of ticket, and a chromosome is an order of tickets. Figure 2b is an example of a chromosome with $K = 10$.

Pareto Set Based Fitness Function: Selecting an appropriate fitness function is very crucial to genetic algorithms (GAs). For instance, the maximin [1] is widely used to achieve optimum performance. However, a set of pareto solutions are generated for each individual in the population of the genetic algorithm of this paper (GA). Therefore, a pareto set based maximin fitness function is presented. Given an objective function f with K objectives, f_k^i denotes k -th objective value of the i -th solution in the specific generation ($k \in \{0, 1\}$ in this paper). The formula of maximin Fitness Function [1] is

$$fitness_i = \max_{j \neq i, j \in X} (\min_{k \in \{0, 1, \dots, K\}} (f_k^i - f_k^j)) \quad (8)$$

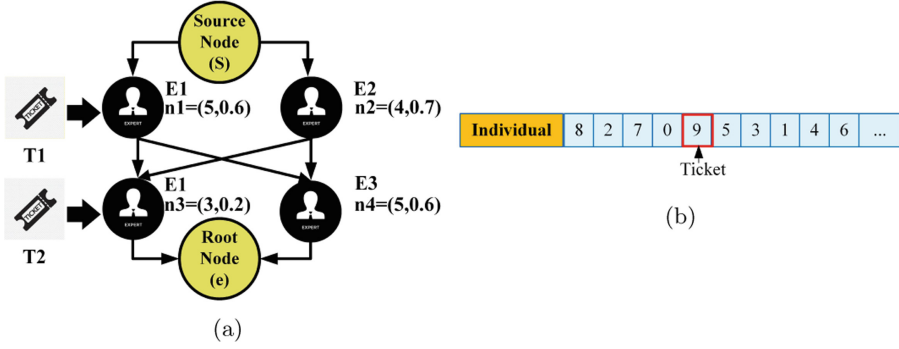


Fig. 2. Examples for MOA* and chromosome encoding

where X is the set of non-dominated solutions. All fitness values are normalized to be in the range of $[0, 1]$ and negative label is employed to transform the maximum of matching degree to a minimum problem coherent with the minimizing of flowtime. The $\min$ is used to find the best objective of i compared with j and $\max_f$ is used to find the worst condition compared with different solution j . If a solution has a maximin fitness value > 0 , which means that it is dominated by others. On the contrary, a maximin fitness value < 0 means the corresponding solution is non-dominated. In existing algorithms for multi-objective optimization, each individual has one corresponding solution. Therefore, the fitness value of each individual can be calculated using Eq. (8) directly. However, for each individual (an order of tickets) in this paper, a set of Pareto solution is obtained by MOA*. Therefore, the fitness of each individual is the average of fitness values of all Pareto solutions. In a population G , there are multiple individuals (chromosomes), and a set of Pareto solutions S_i is obtained by MOA* for each individual $i \in G$. The solutions of all individuals compose a large solution set $S^a = \cup_{i \in G} S_i$. For each solution, the fitness value is calculated in the range of S^a based on Eq. (8). Then, the fitness value of the individual i is obtained by averaging the fitness values of all solutions in S_i .

Selection and Crossover Operation: The Uniform crossover technique is applied in which parents are selected via the roulette wheel selection. For minimizing objectives, individuals with lower maximin fitness values have more chances of being chosen as a parent. Figure 3a demonstrates the process of crossover operation. Initially, two parents named P1 and P2 are selected for crossover as follows. P1 has the minimum fitness value and the roulette wheel method is applied to select P2 randomly. Then, P1's genes are copied to the offspring with a probability of 0.5, and unassigned positions of the offspring are set to be -1 . Next, tickets not appeared in the current offspring are taken from P2 in order generating a temp GeneList. Finally, tickets of the temp GeneList replaced -1 in the offspring chromosome in order obtaining the final complete offspring.

Mutation Operation: For every individual, there is a certain probability to mutate for each gene. The mutation process iterates from the first gene to the last gene one by one. For each gene, there is a probability to select another gene randomly and swap it with the current gene. The Fig. 3b demonstrates an example of mutation process. Firstly, for the first gene, the random value generated by a random number generator is larger than the mutation probability which means the first gene need not to be swapped with others. For the second gene, the random value is generated again and is smaller than the mutation probability. Therefore, the second gene need to be swapped with another randomly selected gene (9 for this step). After genes 2 and 9 are swapped, the current gene changes to the third gene and the above process iterates until the last gene.

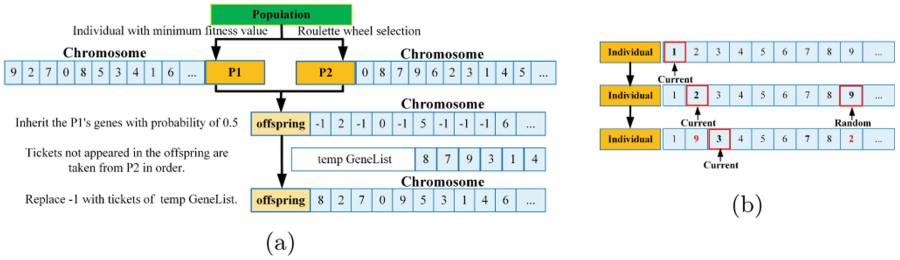


Fig. 3. An illustration of crossover and mutation

5 Performance Evaluation

Since there is no exact algorithms for the considered ticket scheduling problem in terms of flowtime and matching degree, our approach is compared with existing multi-objective NCMCO [1] and IHDPPO [13] algorithms for traditional assignment problems. Algorithms are coded using Java and executed on a Windows system with i5-8400 CPU and 16 GB memory. Algorithms are tested on a randomly generated set of tickets and experts to evaluate their performance. Test data set is divided into three groups named small (30–40 tickets and experts), medium (50–100 tickets and experts), and large (100–150 tickets and experts). Parameters of GAMOA*, NCMCO and IHDPPO are shown in Table 2 which are set to be the same as much as possible for fair comparison. Meanwhile, in order to evaluate the impact of the number of generations to compared algorithms, results of different generations are recorded. Algorithms are given the same computation times. For GAMOA*, results of the 10-th and 20-th generations are recorded because GAMOA* consumes more time to evaluate each individual. On the contrary, NCMCO and IHDPPO run more fast and results of 100-th to 30000-th generations are stored. Then, different generations' results of algorithms are compared pairwise.

Table 2. Parameter settings for GAMOA, NMCMO and IHDPSO

Parameters	Size
Population	100
Crossover probability	0.95
Mutation probability	0.175
GAMOA*'s number of generations (iteration times)	10\20
NMCMO's number of generations	1000\5000\10000\30000
IHDPSO's number of generations	100\1000\5000\20000

The maximum spread and the convergence of two sets [1] are popular metrics for comparing multi-objective algorithms which are defined as follows. **Maximum Spread (MS)** is an Euclidean distance between the maximum and minimum value of the objective function. It is defined to be

$$MS = \sqrt{\sum_{i=1}^K |f_i^{\max} - f_i^{\min}|} \quad (9)$$

where $f_i^{\max}$ and $f_i^{\min}$ denote the maximum and minimum values of each objective, respectively. The higher value of the maximum spread means the vast diversity in the search space, i.e., the maximum spread measures the diversity of a set of solutions. **Coverage of Two Sets (C)** compares the domination of two sets of populations which shows that how better population P' are dominated by population P . The formulation of coverage of two sets is defined to be

$$C(P, P') = \frac{|\{x' \in P'; \exists x \in P, x \text{ dominates } x'\}|}{|P'|} \quad (10)$$

where x and x' are solutions. Larger $C(P, P')$ means more solutions in P' are dominated by solutions in P . For example, when $C(P, P') = 1$, all solutions in P' are dominated by some other solutions in P .

5.1 Experimental Results

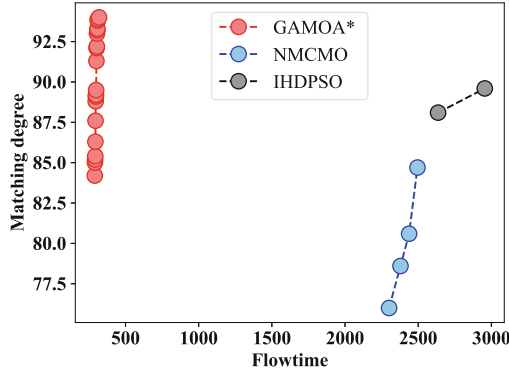
Experimental results in terms of Maximum Spread is shown in Table 3 which denotes that MS of NMCMO are larger than those of GAMOA*. Meanwhile, MS increases as the generation size increases for NMCMO while the generation size has very little impact on MS of GAMOA*. NMCMO is a traditional genetic algorithm and the quality of solutions is diverse, while our approach uses multi-objective A* to find local optimal solutions every time. Therefore, the MS of NMCMO are larger than those of GAMOA*. For example, solutions with too large flowtime have been ignored by GAMOA*, but not by NMCMO. Meanwhile, as the generation size increases, NMCMO finds more different solutions by exploring the search space increasing MS in some degree. IHDPSO obtains

Table 3. Maximum spread of each instance

Group	Tickets-experts	NMC MO(GS)				IHDPSO(GS)				GAMOA*(GS)	
		1000	5000	10000	30000	100	1000	5000	20000	10	20
Small	30-30	11.9	14.3	14.4	15.0	4.1	2.7	3.1	3.8	5.1	5.1
	30-40	13.1	13.1	12.9	15.0	3.2	4.0	5.3	2.7	5.7	5.7
	40-30	15.9	15.9	14.3	15.3	0	17.0	11.9	7.5	6.0	6.0
	40-40	11.1	12.0	13.7	15.6	0	0	5.5	6.9	6.3	6.8
Medium	50-50	11.8	12.2	13.9	15.6	3.5	4.7	0	5.7	6.1	6.1
	50-100	9.6	7.4	13.1	14.0	2.1	2.6	2.8	2.7	4.0	2.0
	100-50	22.6	24.4	24.4	25.4	26.8	30.5	4.4	20.6	7.0	7.2
	100-100	19.8	18.6	19.9	19.1	0	3.5	0	0	6.1	5.1
Large	100-150	14.7	15.1	12.5	18.3	2.6	2.6	0	4.2	4.3	3.3
	150-100	14.2	14.9	15.2	17.5	0	16.4	26.6	14.2	5.2	5.2

Table 4. The coverage of compared algorithms with different generation sizes

Group	GAMOA* (GS)	NMC MO(GS)				IHDPSO(GS)							
		1000	5000	10000	30000	100	1000	5000	20000	1000	5000	20000	30000
Small	10	0.99	0	0.97	0	0.96	0	0.97	0	1	0	1	0
	20	0.99	0	0.97	0	0.96	0	0.97	0	1	0	1	0
Medium	10	0.88	0	0.88	0	0.89	0	0.90	0	0.5	0.22	0.56	0.2
	20	0.89	0	0.89	0	0.91	0	0.91	0	0.5	0.2	0.63	0.17
Large	10	1	0	1	0	1	0	0.97	0	0.25	0.1	0.38	0.14
	20	1	0	1	0	1	0	0.97	0	0.25	0.06	0.38	0.13

**Fig. 4.** Pareto-front comparisons on a large instance

similar MS with GAMOA*, but only gets one solution for many cases leading to a zero MS .

The Coverage $C(GAMOA^*, NMC MO)$ and $C(NMC MO, GAMOA^*)$ are shown in Table 4. $C(GAMOA^*, NMC MO) = 1$ and $C(NMC MO, GAMOA^*) = 0$ for all pairs of different generations which illustrate that

solutions of NCMCO's 30000-*th* generation are even all dominated by solutions of GAMOA*'s 10-*th* generation, and solutions of GAMOA* are all non-dominated by those of NCMCO. In other words, our approach GAMOA* obtains better solutions at early generations taking advantages of multi-objective A*'s effective searching ability. On the contrary, the traditional NCMCO's convergence speed is very slow and it cannot find better solutions even at the 30000-*th* generation. For IHDP SO, $C(\text{GAMOA}^*, \text{IHDP SO})$ is usually much larger than $C(\text{IHDP SO}, \text{GAMOA}^*)$ which means that more solutions of IHDP SO are dominated by those of GAMOA*. As shown in Fig. 4, solutions of GAMOA* dominate all NCMCO's and IHDP SO's solutions, i.e., solutions of our method GAMOA* are all on the Pareto front.

6 Conclusion

In this paper, GAMOA* is proposed to address the multi-objective ticket scheduling problem in large data centers which is a hybrid of the genetic algorithm and multi-objective A star. GAMOA* aims to deal with the multi-objective ticket scheduling to find a set of Pareto expert assignments regarding the total matching degree and the total flowtime. In GAMOA*, the searching effectiveness of traditional genetic algorithm is enhanced by a multi-objective A star algorithm by the aid of transforming ticket scheduling to routing problems. Experimental results illustrate that most solutions found by our approach dominate solutions of NCMCO in terms of the matching degree and the flowtime. Combining ticket routing techniques to enhance our ticket scheduling is a promising future work.

Acknowledgements. This work is supported by the National Natural Science Foundation of China (Grant No. 61972202), the Fundamental Research Funds for the Central Universities (No. 30919011235).

References

1. Alemzadeh, S., Dastghaibiyar, G.: Time and cost trade-off using multi-objective task scheduling in utility grids. In: ICCKE 2013, pp. 362–367. IEEE (2013)
2. Maheswaran, M., Ali, S., Siegel, H.J., Hensgen, D., Freund, R.F.: Dynamic mapping of a class of independent tasks onto heterogeneous computing systems. *J. Parallel Distrib. Comput.* **59**(2), 107–131 (1999)
3. Zhu, L., Li, Q., He, L.: Study on cloud computing resource scheduling strategy based on the ant colony optimization algorithm. *Int. J. Comput. Sci. Issues (IJCSI)* **9**(5), 54 (2012)
4. Gong, M., Jiao, L., Du, H., Bo, L.: Multiobjective immune algorithm with nondominated neighbor-based selection. *Evol. Comput.* **16**(2), 225–255 (2008)
5. Chakravarthy, K., Rajendran, C.: A heuristic for scheduling in a flowshop with the bicriteria of makespan and maximum tardiness minimization. *Prod. Plan. Control* **10**(7), 707–714 (1999)
6. Vidya, G., Sarathambekai, S., Umamaheswari, K., Yamunadevi, S.: Task scheduling using adaptive weighted particle swarm optimization with adaptive weighted sum. *Procedia Eng.* **38**, 3056–3063 (2012)

7. Agarwal, S., Sindhgatta, R., Sengupta, B.: SmartDispatch: enabling efficient ticket dispatch in an it service environment. In: Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 1393–1401 (2012)
8. Shao, Q., Chen, Y., Tao, S., Yan, X., Anerousis, N.: EasyTicket: a ticket routing recommendation engine for enterprise problem resolution. *Proc. VLDB Endow.* **1**(2), 1436–1439 (2008)
9. Sun, P., Tao, S., Yan, X., Anerousis, N., Chen, Y.: Content-aware resolution sequence mining for ticket routing. In: International Conference on Business Process Management, pp. 243–259. Springer, Cham (2010). https://doi.org/10.1007/978-3-642-15618-2_18
10. Izakian, H., Ladani, B.T., Abraham, A., Snasel, V., et al.: A discrete particle swarm optimization approach for grid job scheduling. *Int. J. Innov. Comput. Inf. Control* **6**(9), 1–15 (2010)
11. Sarathambekai, S., Umamaheswari, K.: Task scheduling in distributed systems using heap intelligent discrete particle swarm optimization. *Comput. Intell.* **33**(4), 737–770 (2017)
12. Sarathambekai, S., Umamaheswari, K.: Intelligent discrete particle swarm optimization for multiprocessor task scheduling problem. *J. Algorithms Comput. Technol.* **11**(1), 58–67 (2017)
13. Sarathambekai, S., Umamaheswari, K.: Multi-objective optimization techniques for task scheduling problem in distributed systems. *Comput. J.* **61**(2), 248–263 (2017)
14. Karimi, M.: Hybrid discrete particle swarm optimization for task scheduling in grid computing. *Int. J. Grid Distrib. Comput.* **7**(4), 93–104 (2014)
15. Subashini, G., Bhuvaneswari, M.: Non-dominated particle swarm optimization for scheduling independent tasks on heterogeneous distributed environments. *Int. J. Adv. Soft Comput. Appl.* **3**(1), 1–17 (2011)
16. Subashini, G., Bhuvaneswari, M.C.: Comparison of multi-objective evolutionary approaches for task scheduling in distributed computing systems. *Sadhana* **37**(6), 675–694 (2012). <https://doi.org/10.1007/s12046-012-0102-4>
17. Kardani-Moghaddam, S., Khodadadi, F., Entezari-Maleki, R., Movaghar, A.: A hybrid genetic algorithm and variable neighborhood search for task scheduling problem in grid environment. *Procedia Eng.* **29**, 3808–3814 (2012)
18. Abraham, A., Liu, H., Grosan, C., Xhafa, F.: Nature inspired meta-heuristics for grid scheduling: single and multi-objective optimization approaches. In: Metaheuristics for Scheduling in Distributed Computing Environments, pp. 247–272. Springer, Cham (2008). https://doi.org/10.1007/978-3-540-69277-5_9
19. Pradeep, K., Jacob, T.P.: CGSA scheduler: a multi-objective-based hybrid approach for task scheduling in cloud environment. *Inf. Secur. J. Glob. Perspect.* **27**(2), 77–91 (2018)
20. Mandow, L., Pérez-de-la Cruz, J.-L.: A new approach to multiobjective A* search, pp. 218–223 (2005)